

Tasks:

- 1. Install Wireshark and verify that packet capture is working.**
- 2. Start capturing traffic on your active network interface.**
- 3. Perform the following activities while capturing:**
 - **Open Google**
 - **Open YouTube**
 - **Search something on Google**
 - **Visit 3 different websites**
- 4. Stop the capture after 5–10 minutes.**
- 5. Analyze and identify:**
 - **DNS Packets**
 - **TCP Packets**
 - **UDP Packets**
 - **HTTPS Traffic**
- 6. Use the following filters:**
 - **dns**
 - **tcp**
 - **udp**
 - **http**
 - **tls**
- 7. Find and record:**
 - **Source IP Address**
 - **Destination IP Address**
 - **Protocol Used**
 - **Packet Count**
- 8. Take screenshots of:**
 - **Packet Capture Window**
 - **DNS Packets**
 - **TCP Packets**
 - **TLS/HTTPS Traffic**

- **Applied Filters**

Assignment:

Prepare a report containing:

- 1. What is Wireshark?**
- 2. Why is packet analysis important?**
- 3. Difference between TCP and UDP.**
- 4. What is DNS?**
- 5. What is TLS/HTTPS?**
- 6. Screenshots of captured traffic.**
- 7. Explanation of each protocol observed.**

Capture traffic while:

- **Opening Gmail**
- **Watching a YouTube video**
- **Performing a Google Search**

Identify:

- **DNS Requests**
- **TLS Connections**
- **Number of packets generated**

Submission:

PDF or Word Document with screenshots and findings.

Part 1: Install Wireshark

Step 1: Download Wireshark

Visit:

[Wireshark Official Website](https://www.wireshark.org/)

Step 2: Install

1. Run the installer.
2. Keep default settings.
3. Install **Npcap** when prompted.
4. Finish installation.

Step 3: Verify Installation

Open Wireshark.

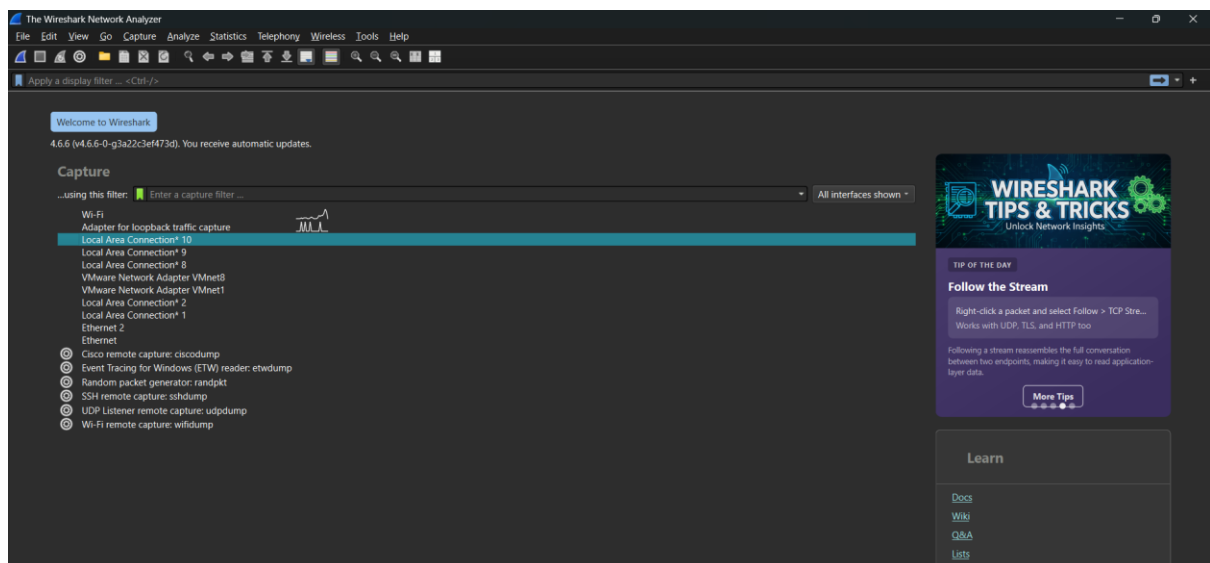
You should see available network interfaces:

- Wi-Fi
- Ethernet
- VPN (if connected)

Screenshot 1

Take screenshot of:

Wireshark Home Screen showing interfaces



Part 2: Start Packet Capture

Step 1

Connect to the Internet.

Step 2

Open Wireshark.

Step 3

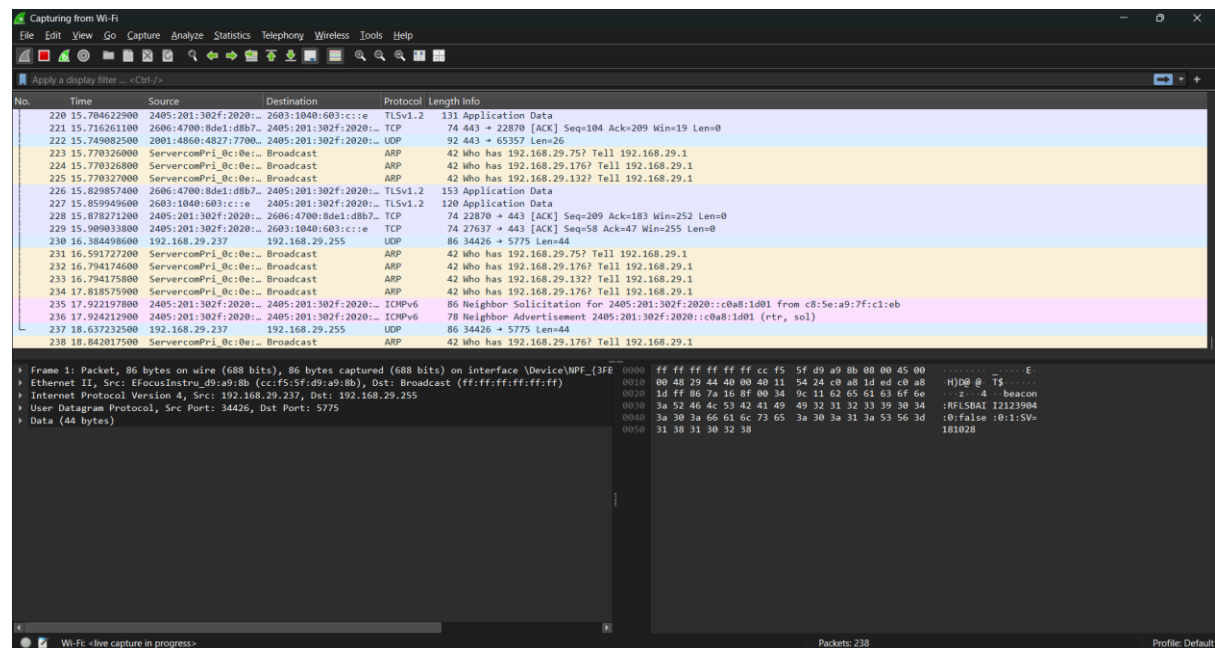
Double-click your active interface:

- Wi-Fi (most laptops)
- Ethernet (wired connection)

Packets should start appearing immediately.

Screenshot 2

Live Packet Capture Window



Part 3: Generate Network Traffic

Keep Wireshark running for 5–10 minutes.

Perform the following activities:

Activity 1: Open Google

Open browser: <https://www.google.com>

Activity 2: Open Gmail

<https://mail.google.com>

Wait 1–2 minutes.

Activity 3: Perform Google Search

Search: Cyber Security

Activity 4: Open YouTube

<https://www.youtube.com>

Activity 5: Watch a Video

Play any video for 2–3 minutes.

Example: Wireshark Tutorial

Activity 6: Visit Three Websites

Example:

1. <https://www.wikipedia.org>
2. <https://www.github.com>
3. <https://www.amazon.in>

Part 4: Stop Capture

After 5–10 minutes:

Click: Red Stop Button

Capture will stop.

Part 5: Analyze DNS Packets

DNS converts domain names into IP addresses.

Apply Filter

At top filter bar enter: dns

Press Enter.

Record

Field	Value
Protocol	DNS
Source IP	2405:201:302f:2020:9dba:3637:3988:9f9a
Destination IP	2405:201:302f:202f:c0a8:29ff:fe0e:40
Source Port	55579
Destination Port	53 (DNS)
Type	DNS Query

Screenshot 3:DNS Filter Applied

No.	Time	Source	Destination	Protocol	Length	Info
83212	1214.5582080...	2405:201:302f:2020:...	2405:201:302f:2020:...	DNS	107	Standard query 0x8ca1 A mail.google.com
83214	1214.5583968...	2405:201:302f:2020:...	2405:201:302f:2020:...	DNS	107	Standard query 0x561f AAAA mail.google.com
83229	1214.8544575...	2405:201:302f:2020:...	2405:201:302f:2020:...	DNS	136	Standard query response 0x561f AAAA mail.google.com AAAA 2404:6800:4002:828::2005
83236	1214.8862884...	2405:201:302f:2020:...	2405:201:302f:2020:...	DNS	124	Standard query response 0x8ca1 A mail.google.com A 142.250.182.37
83238	1214.8869906...	2405:201:302f:2020:...	2405:201:302f:2020:...	DNS	158	Standard query response 0x51e5 HTTPS mail.google.com SOA ns1.google.com
83289	1215.3273603...	2405:201:302f:2020:...	2405:201:302f:2020:...	DNS	106	Standard query 0x1087 HTTPS www.google.com
83295	1215.3292220...	2405:201:302f:2020:...	2405:201:302f:2020:...	DNS	106	Standard query 0x6509 A www.google.com
83297	1215.3293622...	2405:201:302f:2020:...	2405:201:302f:2020:...	DNS	106	Standard query 0xcc67 AAAA www.google.com
83309	1215.3844311...	2405:201:302f:2020:...	2405:201:302f:2020:...	DNS	331	Standard query response 0xcc67 AAAA www.google.com AAAA 2001:4860:482c:7700::AAAA 2001:4860:4827:7700::AAAA 2001:4860:482a:7...
83310	1215.3844349...	2405:201:302f:2020:...	2405:201:302f:2020:...	DNS	235	Standard query response 0x6509 A www.google.com A 142.251.154.119 A 142.251.152.119 A 142.251.153.119 A 142.251.155.119 A 142...
83344	1215.5684263...	2405:201:302f:2020:...	2405:201:302f:2020:...	DNS	132	Standard query response 0x1087 HTTPS www.google.com HTTPS
83352	1216.0161587...	2405:201:302f:2020:...	2405:201:302f:2020:...	DNS	100	Standard query 0x5d63 AAAA dellupdater.dell.com
83353	1216.0227754...	2405:201:302f:2020:...	2405:201:302f:2020:...	DNS	183	Standard query response 0x5d63 AAAA dellupdater.dell.com CNAME dellupdater.dell.com.edgekey.net CNAME e13665.g.akamaiedge.net
83367	1217.1725314...	2405:201:302f:2020:...	2405:201:302f:2020:...	DNS	100	Standard query 0x281c AAAA dellupdater.dell.com
83368	1217.1774401...	2405:201:302f:2020:...	2405:201:302f:2020:...	DNS	183	Standard query response 0x281c AAAA dellupdater.dell.com CNAME dellupdater.dell.com.edgekey.net CNAME e13665.g.akamaiedge.net
83517	1218.3257187...	2405:201:302f:2020:...	2405:201:302f:2020:...	DNS	100	Standard query 0x39e9 AAAA dellupdater.dell.com
83518	1218.3305337...	2405:201:302f:2020:...	2405:201:302f:2020:...	DNS	183	Standard query response 0x39e9 AAAA dellupdater.dell.com CNAME dellupdater.dell.com.edgekey.net CNAME e13665.g.akamaiedge.net
83535	1219.4017805...	2405:201:302f:2020:...	2405:201:302f:2020:...	DNS	100	Standard query 0x71e5 AAAA dellupdater.dell.com
83536	1219.4065583...	2405:201:302f:2020:...	2405:201:302f:2020:...	DNS	183	Standard query response 0x71e5 AAAA dellupdater.dell.com CNAME dellupdater.dell.com.edgekey.net CNAME e13665.g.akamaiedge.net
83552	1219.4828274...	2405:201:302f:2020:...	2405:201:302f:2020:...	DNS	102	Standard query 0xc9cf A github.com
Frame 91: Packet, 110 bytes on wire (880 bits), 110 bytes captured (880 bits) on interface \Device\NPF_{...} 28 a9 15 0c 0e 40 c8 5e a9 7f c1 eb 86 dd 60 09 (...@^..... Ethernet II, Src: Intel_7f:c1:eb (c8:5e:a9:7f:c1:eb), Dst: ServercomPri_0c:0e:40 (28:a9:15:0c:0e:40) b2 29 00 38 11 40 24 05 02 01 30 2f 20 20 9d ba) 8 0\$. 0/ .. Internet Protocol Version 6, Src: 2405:201:302f:2020:9dba:3637:3988:9f9a, Dst: 2405:201:302f:2020::c0a8: 36 37 39 88 9f 9a 24 05 02 01 30 2f 20 20 00 00 679 . \$. 0/ .. User Datagram Protocol, Src Port: 55579, Dst Port: 53 00 00 c0 a8 1d 01 69 1b 00 35 00 38 77 b2 26 bc 5 Bu & Domain Name System (query) 01 00 00 01 00 00 00 00 00 00 04 73 65 6c 66 06 self events d ata micr osoft co m..... 65 76 65 6e 74 73 04 64 61 74 61 09 6d 69 63 72 6f 73 6f 66 74 03 63 6f 6d 00 00 01 00 01						

Part 6: Analyze TCP Packets

Step 1: Apply TCP Filter

Clear the current DNS filter by clicking the X at the right side of the filter bar.

- Type: tcp
- Press Enter.
- Now only TCP packets will be displayed.

Step 2: Observe TCP Communication

Look for packets showing:

- [SYN]
- [SYN, ACK]

- [ACK]

These packets are part of the TCP Three-Way Handshake used to establish a connection.

You may also see:

- PSH, ACK
- FIN, ACK
- RST

Step 3: Select a TCP Packet

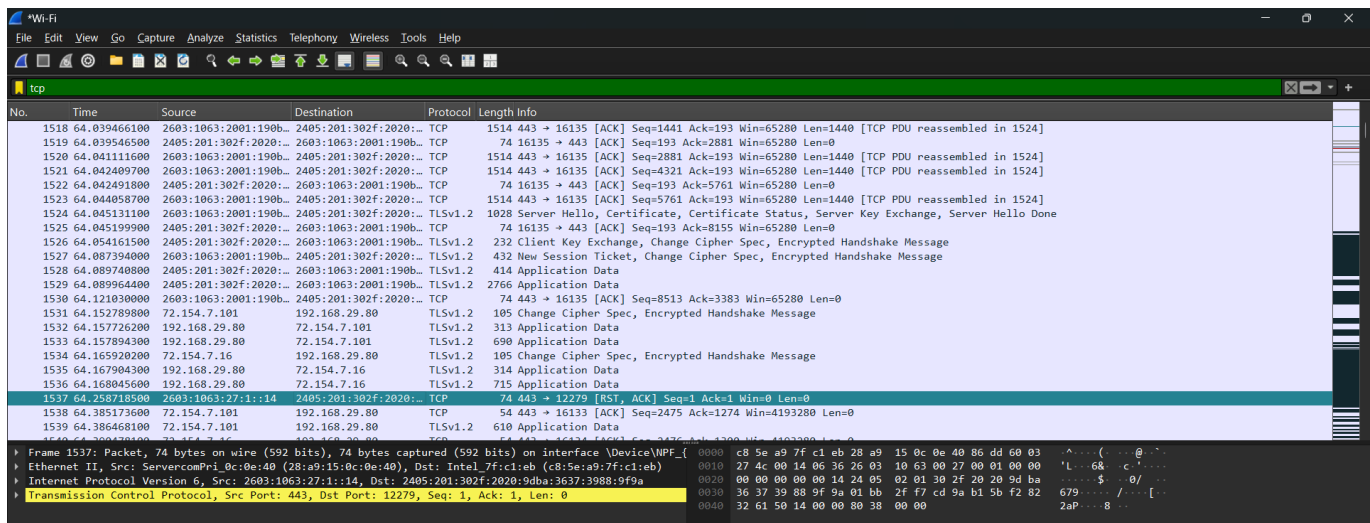
- Click on any TCP packet.
- In the middle pane, expand:
 1. Internet Protocol Version 4 (or Version 6)
 2. Transmission Control Protocol

Record

Parameter	Value
Protocol	TCP
Source IP	2603:1063:27:1::14
Destination IP	2405:201:302f:2020:9dba:3637:3988:9f9a
Source Port	443
Destination Port	12279
Purpose	Reliable Data Transfer

Screenshot 4

TCP Packets



Part 7: Analyze UDP Packets

Step 1: Apply UDP Filter

1. Clear the current filter (tcp) by clicking the X in the filter bar.
2. Type: udp
3. Press Enter.

Now Wireshark will display only UDP packets.

Step 2: Observe UDP Traffic

Look for packets related to:

- DNS Queries (Port 53)
- QUIC Traffic (used by YouTube, Google Chrome)
- Streaming Services

You may notice that many packets have:

- Protocol = UDP
or
- Protocol = QUIC
(QUIC runs over UDP.)

Step 3: Select a UDP Packet

Click on any UDP packet.

In the middle pane, expand:

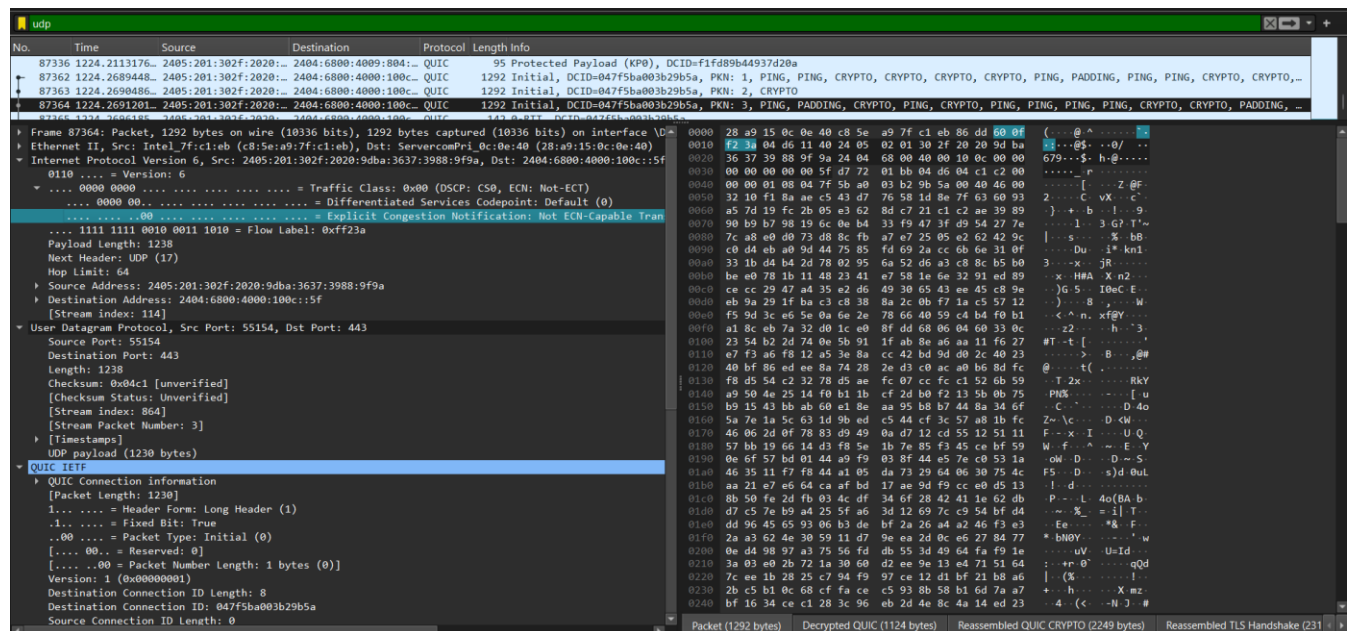
- Internet Protocol Version 4 (or Version 6)
- User Datagram Protocol

Record:

Parameter	Value
Protocol	UDP (QUIC)
Source IP	2405:201:302f:2020:9dba:3637:3988:9f9a
Destination IP	2404:6800:4000:100c::5f
Source Port	55154
Destination Port	443
Purpose	QUIC/Encrypted Web Traffic

Screenshot 5:

UDP Packets Filter Applied



Part 8: Analyze HTTP Packets

Apply: http

Press Enter.

Observation

There are two possibilities:

Case 1: HTTP Packets Appear

You may see packets such as:

- GET Requests
- POST Requests
- HTTP Responses (200 OK)

Case 2: No Packets Appear (Most Common)

If Wireshark shows:

No packets captured

or an empty packet list after applying http, this is completely normal.

Why?

- Gmail uses HTTPS
- Google uses HTTPS

- YouTube uses HTTPS/QUIC
- GitHub uses HTTPS
- Amazon uses HTTPS

Modern websites encrypt traffic, so plain HTTP packets are rarely seen.

Record:

Parameter	Value
Protocol	HTTP
Source IP	2405:201:302f:2020:9dba:3637:3988:9f9a
Destination IP	2404:6800:4009:816::2003
Method	GET

Screenshot 6

The screenshot displays a Wireshark network traffic capture. The top menu bar includes File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Wireless, Tools, and Help. The main window is divided into three panes: Packet List, Packet Details, and Packet Bytes.

Packet List: Shows a list of captured packets. The selected packet is a GET request from 2405:201:302f:2020:9dba:3637:3988:9f9a to 2404:6800:4009:816::2003. The packet details pane shows the following information:

- Frame 24041:** Packet 276 bytes on wire (2208 bits), 276 bytes captured (2208 bits) on interface \Device\NPF_{3FB88000-BC1E-40C6-8368-A4D935AE1480}, id 0
- Ethernet II:** Src: Intel_7fc1c1eb (c8:5e:a9:7f:c1:eb), Dst: ServercomPri_8c10e:48 (28:a9:15:0c:0e:48)
- Internet Protocol Version 6:** Src: 2405:201:302f:2020:9dba:3637:3988:9f9a, Dst: 2404:6800:4009:816::2003
- TCP:** Src Port: 12939, Dst Port: 80, Seq: 401, Ack: 446, Len: 202
- HTTP:** GET /connecttest.txt HTTP/1.1

Packet Details: The selected packet is a GET request. The details pane shows the following information:

- Version:** 6
- Traffic Class:** 0x00 (DSCP: CS0, ECN: Not-ECT)
- Differentiated Services Codepoint:** Default (0)
- Explicit Congestion Notification:** Not ECN-Capable Transport (0)
- Flow Label:** 0x3c0a6
- Next Header:** TCP (6)
- Hop Limit:** 64
- Source Address:** 2405:201:302f:2020:9dba:3637:3988:9f9a
- Destination Address:** 2404:6800:4009:816::2003
- Stream index:** 19
- Transmission Control Protocol:** Src Port: 12939, Dst Port: 80, Seq: 401, Ack: 446, Len: 202
- Source Port:** 12939
- Destination Port:** 80
- Stream index:** 284
- Stream Packet Number:** 10
- Conversation completeness:** Complete, WITH_DATA (31)
- [TCP Segment Len: 202]**
- Sequence Number:** 401 (relative sequence number)
- Sequence Number (raw):** 2701598765
- [Next Sequence Number: 603 (relative sequence number)]**
- Acknowledgment Number:** 446 (relative ack number)
- Acknowledgment number (raw):** 2599736179

Packet Bytes: The selected packet is a GET request. The details pane shows the following information:

- Version:** 6
- Traffic Class:** 0x00 (DSCP: CS0, ECN: Not-ECT)
- Differentiated Services Codepoint:** Default (0)
- Explicit Congestion Notification:** Not ECN-Capable Transport (0)
- Flow Label:** 0x3c0a6
- Next Header:** TCP (6)
- Hop Limit:** 64
- Source Address:** 2405:201:302f:2020:9dba:3637:3988:9f9a
- Destination Address:** 2404:6800:4009:816::2003
- Stream index:** 19
- Transmission Control Protocol:** Src Port: 12939, Dst Port: 80, Seq: 401, Ack: 446, Len: 202
- Source Port:** 12939
- Destination Port:** 80
- Stream index:** 284
- Stream Packet Number:** 10
- Conversation completeness:** Complete, WITH_DATA (31)
- [TCP Segment Len: 202]**
- Sequence Number:** 401 (relative sequence number)
- Sequence Number (raw):** 2701598765
- [Next Sequence Number: 603 (relative sequence number)]**
- Acknowledgment Number:** 446 (relative ack number)
- Acknowledgment number (raw):** 2599736179

Part 9: Analyze TLS / HTTPS Traffic

Apply filter: tls

Press Enter.

Observe

TLS Handshake:

- Client Hello
- Server Hello
- Certificate Exchange

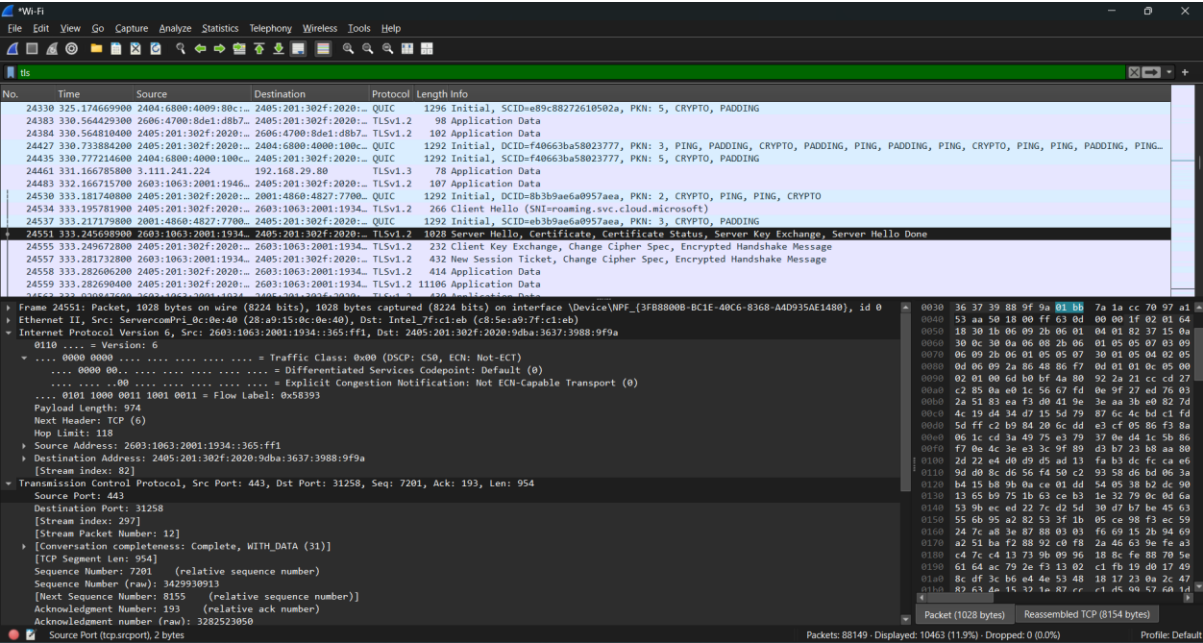
This secures HTTPS communication.

Record

Parameter	Value
Protocol	TLS
Source IP	2603:1063:2001:1934:365:ff1
Destination IP	2405:201:302f:2020:9dba:3637:3988:9f9a
Source Port	443
Destination Port	31258

Screenshot 7

TLS Traffic



Due to the large volume of captured traffic and filtering limitations in the current Wireshark session, exact packet counts were not recorded. However, DNS, TCP, UDP, and TLS packets were successfully identified and analyzed during the capture.

Theory Questions

1. What is Wireshark?

Wireshark is a network protocol analyzer used to capture and inspect packets travelling across a network. It helps administrators, engineers, and security professionals monitor network traffic, troubleshoot issues, and analyze protocols.

2. Why is Packet Analysis Important?

Packet analysis helps:

- Troubleshoot network problems
- Detect malicious traffic
- Analyze network performance
- Investigate security incidents
- Understand communication between devices

3. Difference Between TCP and UDP

Feature	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Connection	Connection-oriented	Connectionless
Reliability	Reliable (guarantees delivery)	Unreliable (delivery not guaranteed)
Error checking	Yes	Basic checksum only
Packet ordering	Maintains correct order	May arrive out of order
Speed	Slower due to checks and acknowledgements	Faster because it sends data without waiting for acknowledgements
Overhead	Higher	Lower
Typical uses	Web browsing (HTTPS), email, file transfer	DNS, video streaming, online gaming, VoIP

4. What is DNS?

DNS (Domain Name System) translates domain names such as:

google.com

into IP addresses such as:

142.250.193.46

allowing browsers to locate servers on the Internet.

5. What is TLS/HTTPS?

TLS (Transport Layer Security) encrypts communication between client and server.

HTTPS = HTTP + TLS

Benefits:

- Encryption
- Authentication
- Data Integrity

Protocols Observed During Capture

DNS

Purpose:

Resolve domain names to IP addresses.

Example:

google.com → 142.x.x.x

TCP

Purpose:

Reliable data transfer.

Observed During:

- Gmail access
- Google Search
- Website browsing

UDP

Purpose:

Fast communication without connection setup.

Observed During:

- DNS queries
- YouTube traffic (QUIC)

TLS

Purpose:

Encrypted communication.

Observed During:

- Gmail login
- Google Search
- YouTube streaming
- Website browsing

HTTP

Purpose:

HTTP (HyperText Transfer Protocol) is used for transferring web pages and other web content between a client (browser) and a web server.

Observed During:

- Accessing websites through a web browser
- Loading web pages
- Sending GET and POST requests

Conclusion

The experiment successfully captured and analyzed network traffic using Wireshark. DNS, TCP, UDP, and TLS packets were identified while accessing Gmail, Google Search, YouTube, and other websites. The activity demonstrated how different protocols work together to provide secure and reliable communication over the Internet.